

SISTEMAS OPERATIVOS CS3015 · 2026-2 · PROYECTO 1

Hilos POSIX como librería de usuario

estudio e implementación del trabajo de Mueller (1993)

Artículo: F. Mueller, *A Library Implementation of POSIX Threads under UNIX*, Winter USENIX 1993

Dario Ricardo Nuñez Villacorta

UTEC · Profesores: Jorge Gonzalez, Mariana Capuñay · TA: Sofía Ku

El contexto: 1992, POSIX y máquinas sin hilos

- ▶ Se estaba votando el borrador POSIX 1003.4a (*Pthreads*).
- ▶ Pedía mucho más que crear hilos: prioridades con apropiación, señales por hilo, mutex, variables de condición, cancelación.
- ▶ SunOS 4 en SPARC **no tenía hilos en el kernel**.
- ▶ Mueller construye todo como una **librería de C en modo usuario**, sin tocar UNIX.

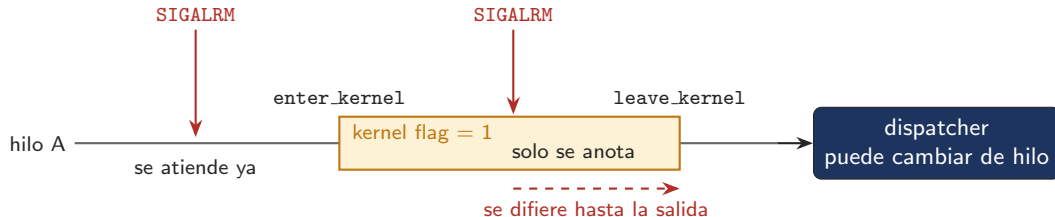
Opción	Costo principal
Kernel	cada operación entra y sale del kernel
Librería	dos planificadores y señales difíciles
Mixta	SunOS, LynxOS

Tesis del paper

Una implementación solo de librería es factible y puede ser tan rápida o más que las que usan el kernel.

La idea central: la librería tiene su propio “kernel”

Una zona de código que corre en exclusión mutua. Las colas y los bloques de control solo se tocan ahí.



Dos variables bastan

kernel flag: estoy dentro.

dispatcher flag: al salir, hay que despachar.

Por qué un solo candado

Es un uniprocador: el grano grueso no pierde concurrencia y ahorra operaciones.

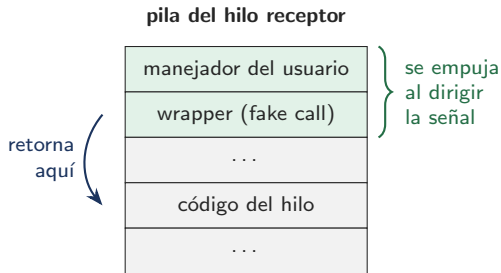
Señales por hilo: fake calls

Problema. El manejador que registró un hilo debe correr *a la prioridad de ese hilo*, pero la señal puede llegar mientras corre cualquier otro.

Solución. No llamarlo. Empujar sobre la pila del hilo receptor un marco que simula una llamada (*wrapper*):

1. el hilo es despachado,
2. el wrapper corre el manejador,
3. vuelve al punto exacto de interrupción.

Extra: antes de volver a un hilo interrumpido se enmascaran las señales, para que no se apilen manejadores sin límite.



Inversión de prioridades: herencia vs. techo

P_1 (baja) toma un mutex, P_3 (alta) lo pide y se bloquea, P_2 (media) corre en su lugar.

Sin protocolo

P_2 no usa el recurso pero retrasa a P_3 tanto como quiera.

espera no acotada

Herencia

Cuando P_3 se bloquea, P_1 sube a la prioridad de P_3 .

al soltar: búsqueda lineal

Techo (SRP)

Al tomar el mutex, P_1 sube al techo fijo del mutex.

al soltar: *pop* de una pila

- ▶ El techo necesita menos cambios de contexto y retiene el mutex menos tiempo.
- ▶ La herencia se adapta a prioridades que cambian en ejecución.
- ▶ Mezclar ambos en secciones anidadas rompe la disciplina de pila del techo (Tabla 4).

Perverted scheduling y resultados

Depurar forzando conmutaciones

- ▶ **Mutex switch:** cambiar de hilo en cada lock exitoso.
- ▶ **Round-robin ordered:** cambiar al salir del kernel.
- ▶ **Random switch:** cambiar al azar, con semilla.

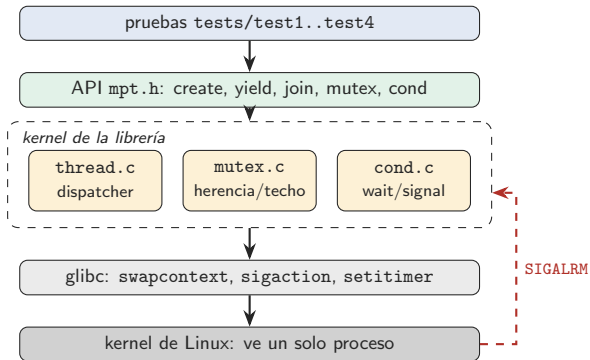
Deterministas: la misma semilla repite el entrelazado. Así encontraron errores del runtime de Ada que FIFO escondía.

Mediciones (SPARC IPX, μs)

Entrar/salir kernel de la librería	0.4
Entrar/salir kernel UNIX (getpid)	18
Cambio de contexto entre hilos	37
Cambio de contexto entre procesos	123
Señal a un hilo, interna	52
Señal a un hilo, externa	250

~45× más barato entrar a la librería. El punto débil: señales externas, que pagan el manejador de UNIX más la redistribución.

Lo que implementé



- ▶ C puro, ~1000 líneas, sin ensamblador.
- ▶ Linux (WSL) en x86-64.
- ▶ Todo lo que toca colas pasa por `mpt__enter_kernel` y `mpt__leave_kernel`.
- ▶ Apropiación con `setitimer` + `SIGALRM`.
- ▶ Mutex con tres protocolos: ninguno, herencia, techo.
- ▶ Tres políticas perversas con semilla.

De SPARC 1993 a Linux 2026

Artículo (SunOS 4, SPARC)

mini-pthreads

Volcar *register windows*
(`ST_FLUSH_WINDOWS`)

`swapcontext`: guarda registros y máscara de señales

Kernel flag y dispatcher flag

`g_kernel_flag`, `g_dispatcher_flag` (`volatile sig_atomic_t`)

Manejador universal

`universal_signal_handler()`: dentro del kernel solo anota

Colas de listos por prioridad

una sola lista, recorrida buscando la mayor prioridad

Mutex con `ldstub` en secuencia
atómica reinicializable

sin instrucción atómica: el lock ocurre dentro del kernel de la librería

Fake call en la pila del hilo

contexto auxiliar con pila propia (prueba 4)

Lo esencial se conserva

guardar contextos en modo usuario, apropiarse con una señal y proteger las estructuras de esa señal. Lo que cambia es propio de la máquina de la época.

El corazón: salir del kernel

```
for (;;) {
    if (!g_dispatcher_flag) {
        g_kernel_flag = 0; return;
    }
    g_dispatcher_flag = 0;
    /* ¿sigo yo o hay alguien mejor? */
    ...
    next = pick_next();
    if (next != g_current)
        do_switch(g_current, next);

    /* ya en el hilo que recibió el control */
    if (!g_deferred_signal) {
        g_kernel_flag = 0; return;
    }
    g_deferred_signal = 0; /* reintentar */
}
```

`mpt_leave_kernel()` en `thread.c`, recortado

El flag se baja después del cambio

Mientras se hace `swapcontext` el kernel flag sigue en 1. Si llega `SIGALRM` justo ahí, se difiere en lugar de empezar un despacho encima de otro.

Reintento

Si llegó una señal dentro del kernel, se vuelve a despachar: puede cambiar quién debe correr. Es el bucle de la Figura 2 del paper.

Prueba 1 y prueba 3: el núcleo funciona

test1: productor y consumidor

```
root@LAPTOP-03D5EE88:/mnt/c/Users/Dario/Desktop/P1_S0# ./bin/test1_monitor
test1_monitor: OK
```

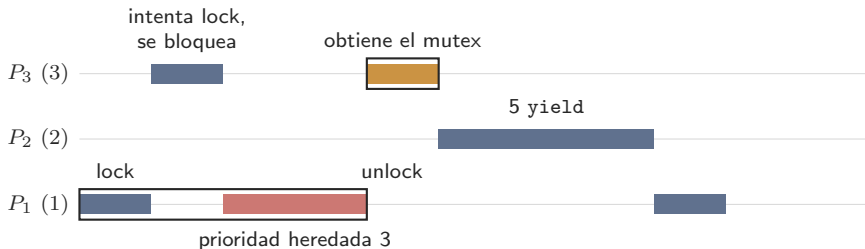
- ▶ El consumidor espera en `cond_wait`; el productor señala.
- ▶ Liberar el mutex y bloquearse ocurren juntos dentro del kernel.
- ▶ El `while` reevalúa el predicado al despertar, como pide el paper.

test3: cinco hilos, ciclo completo

```
root@LAPTOP-03D5EE88:/mnt/c/Users/Dario/Desktop/P1_S0# ./bin/test3_perverted
test3_perverted: OK
```

- ▶ Cada hilo se despacha una vez y termina.
- ▶ trampoline llama a `mpt_exit` al retornar.
- ▶ Con la cola vacía, el control vuelve a `main`.

Prueba 2: la Figura 5(b) con herencia



```
root@LAPTOP-03D5EE88: /mnt/c/Users/Dario/Desktop/P1_S0# ./bin/test2_inversion
test2_inversion: OK
```

- ▶ P_1 crea a P_3 dentro de su sección crítica; P_3 lo desplaza, pide el mutex y le hereda prioridad 3.
- ▶ P_2 se crea después, pero con prioridad 2 ya no desplaza a P_1 .
- ▶ P_2 **no corre ni una vez mientras P_3 espera.**

Prueba 4: fake calls con SIGALRM real

```
root@LAPTOP-03D5EE88:/mnt/c/Users/Dario/Desktop/P1_SO# ./bin/test4_fakecall
Demo 4: fake calls (Figura 3 del paper de Mueller)
2 hilos cooperativos; SIGALRM real cada 150 ms, dirigida por turnos a un hilo distinto del que este corriendo

hilo 0: paso 1/5 (prioridad 1)
hilo 1: paso 1/5 (prioridad 2)
hilo 0: paso 2/5 (prioridad 1)
hilo 1: paso 2/5 (prioridad 2)
[SIGALRM #1] llega mientras corre hilo 1 -> dirigida al hilo 0 (que NO esta corriendo): se DIFIERE
-> [fake call] hilo 0 ejecuta AHORA su handler para la signal 14 (prioridad 1), justo antes de reanudar su codigo
(handler de usuario) hilo 0 atiende la señal 14: actualiza su estado interno y sigue
<- [fake call] hilo 0 retorna exactamente al punto donde habia quedado
hilo 0: paso 3/5 (prioridad 1)
hilo 1: paso 3/5 (prioridad 2)
hilo 0: paso 4/5 (prioridad 1)
hilo 1: paso 4/5 (prioridad 2)
[SIGALRM #2] llega mientras corre el propio hilo 1 -> se difiere igual, se atendera en su proximo despacho
hilo 0: paso 5/5 (prioridad 1)
-> [fake call] hilo 1 ejecuta AHORA su handler para la signal 14 (prioridad 2), justo antes de reanudar su codigo
(handler de usuario) hilo 1 atiende la señal 14: actualiza su estado interno y sigue
<- [fake call] hilo 1 retorna exactamente al punto donde habia quedado
hilo 1: paso 5/5 (prioridad 2)
hilo 0: termino
hilo 1: termino
[SIGALRM #3] llega mientras corre hilo 1 -> dirigida al hilo 0 (que NO esta corriendo): se DIFIERE
test4_fakecall: OK
```

Señal #1: va al hilo 0, que no corre. Se marca; en su siguiente turno el wrapper ejecuta el manejador y luego sigue con el paso 3.

Señal #2: va al propio hilo 1. Tampoco se ejecuta en el acto: espera su próximo despacho.

Señal #3: llega con todo terminado y nunca se atiende. Un hilo terminado no se vuelve a despachar.

Simplificación: el wrapper usa una pila auxiliar; con `ucontext` no hay forma limpia de empujar un marco en una pila en uso.

Lo que encontré al comparar con el paper

A favor

- ▶ **La cola única simplifica la herencia.** Subir la prioridad del dueño no obliga a reubicarlo: `pick_next` lee la prioridad al elegir.
- ▶ **Sin atómicas en uniprosesor.** Si el lock pasa por el kernel de la librería, el flag basta; la única concurrencia es la señal y la señal se difiere.

Diferencias pendientes

- ▶ `do_switch` **desenmascara** `SIGALRM`; el paper la enmascara al volver a un hilo interrumpido.
- ▶ La herencia vuelve a `base_prio`: con mutex anidados podría reaparecer la inversión.
- ▶ Con techo, un `yield` dentro de la sección crítica deja pasar a P_3 , que se bloquea igual.
- ▶ Pilas de hilos terminados sin liberar; `printf` en el manejador de la prueba 4.

Conclusiones

- ▶ **Factible:** con `ucontext`, `sigaction` y `setitimer` hay hilos con prioridades, apropiación, mutex contra la inversión y señales diferidas, sin que Linux lo sepa.
- ▶ **La idea clave es el kernel de la librería:** dos variables convierten una señal que puede caer en cualquier instrucción en algo que solo actúa en puntos seguros.
- ▶ **Las señales son lo difícil**, igual que dice Mueller: ahí están mis desviaciones del paper y ahí su propio punto débil en rendimiento.
- ▶ **Pendiente:** medir tiempos contra `getpid`, enmascarar al volver a hilos interrumpidos, pila de prioridades por hilo y pruebas con apropiación y políticas perversas.

¿Preguntas?